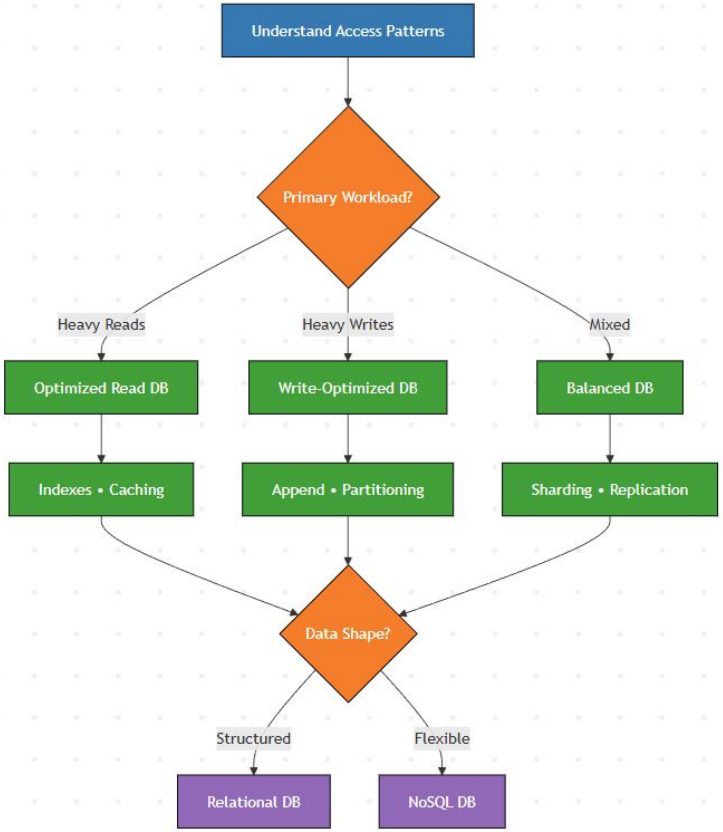
Storage & Databases

System Design Crash Course: Quick Revision Before Interviews

Rahul Rajat Singh

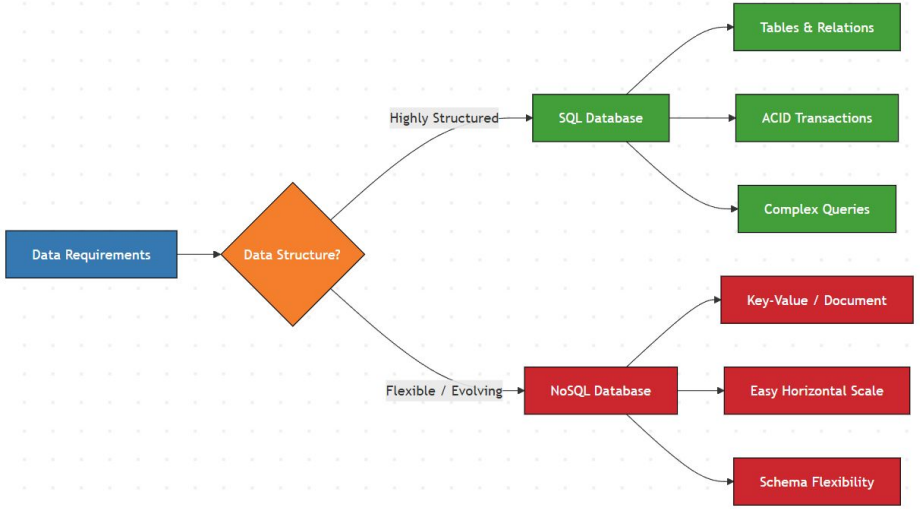
Choosing the Right Database

- Access patterns drive database choice
- Reads vs writes vs queries
- Structured vs flexible data
- Consistency vs availability needs
- Scale path matters early



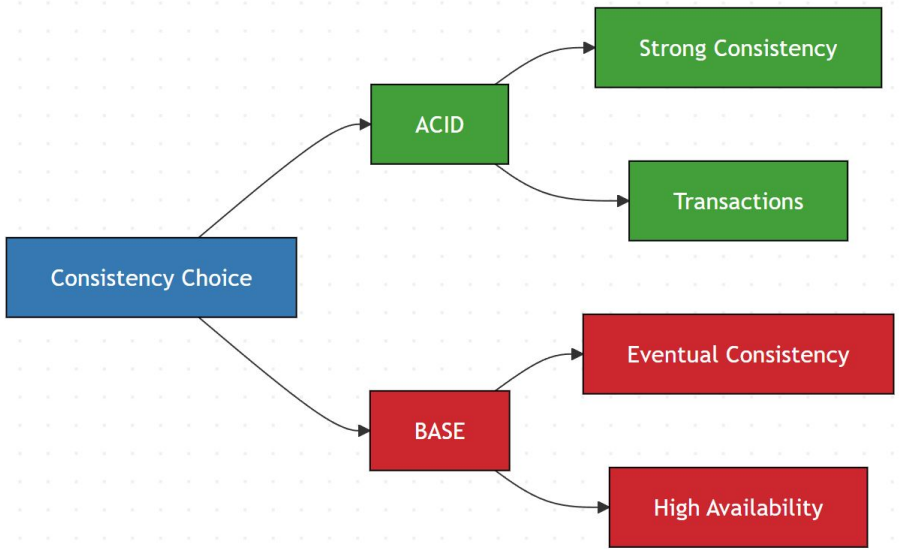
SQL vs NoSQL

- SQL: structured schema, relational data
- NoSQL: flexible schema, varied models
- SQL favors consistency and joins
- NoSQL favors scale and agility
- Choice driven by access patterns



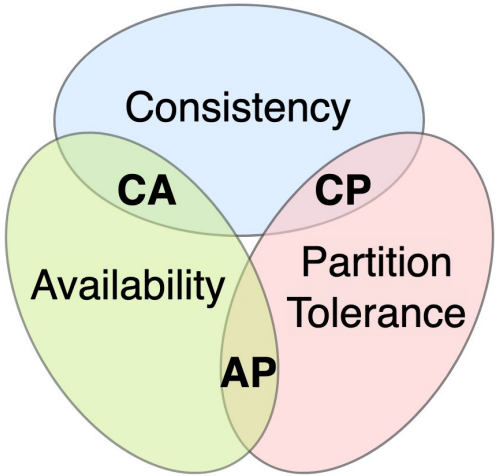
ACID vs BASE

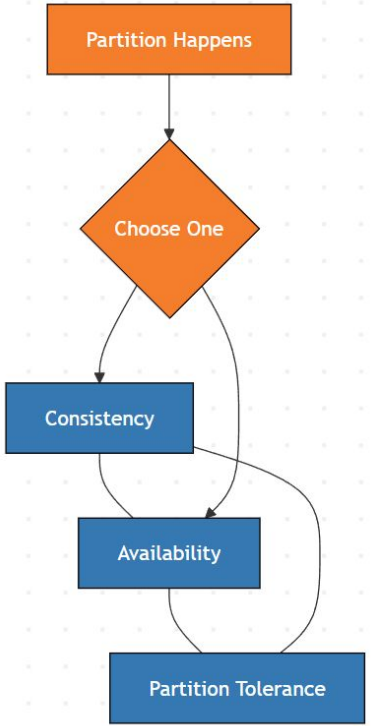
- ACID: strong transactional guarantees
- BASE: eventual consistency model
- ACID favors correctness
- BASE favors availability and scale
- Choice depends on failure tolerance



CAP Theorem (Practical View)

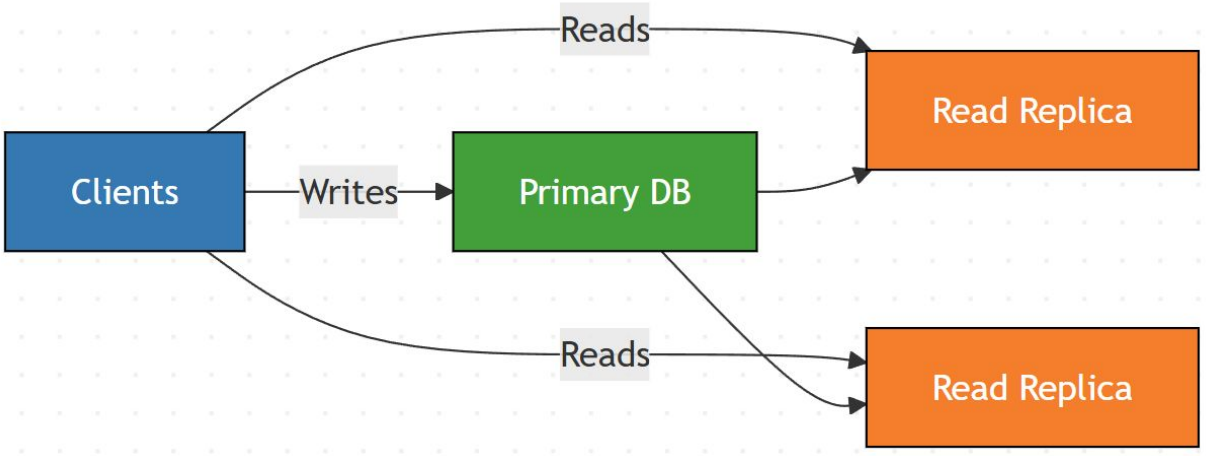
- Consistency, Availability, Partition tolerance
- Network partitions are inevitable
- Must choose between C or A
- CAP applies during failures only
- Design choice, not database feature





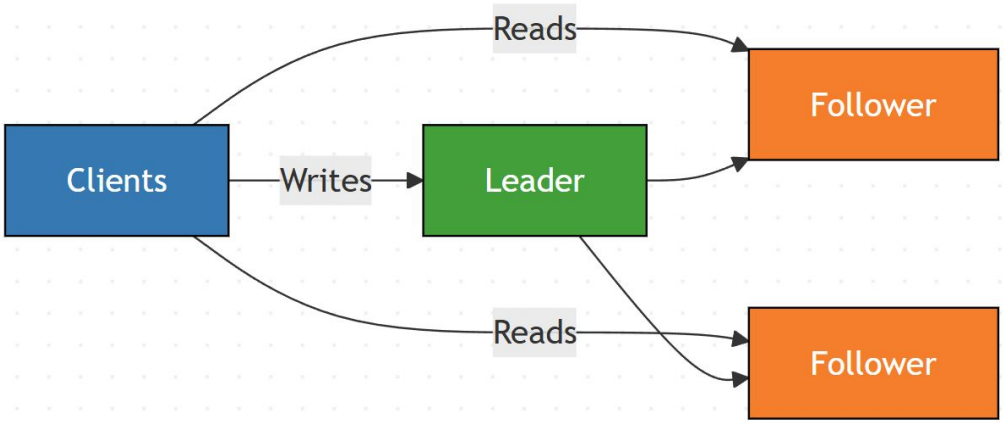
Replication

- Copy data across multiple nodes
- Primary handles writes
- Replicas serve read traffic
- Improves read scalability
- Risk of stale reads



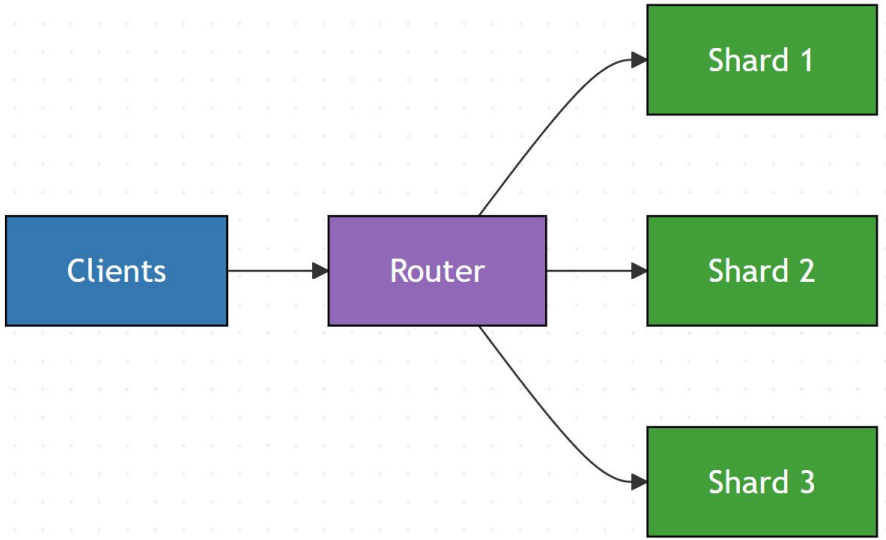
Leader–Follower Replication

- Single leader handles all writes
- Followers replicate leader data
- Reads served from followers
- Simplifies write consistency
- Leader failure needs failover



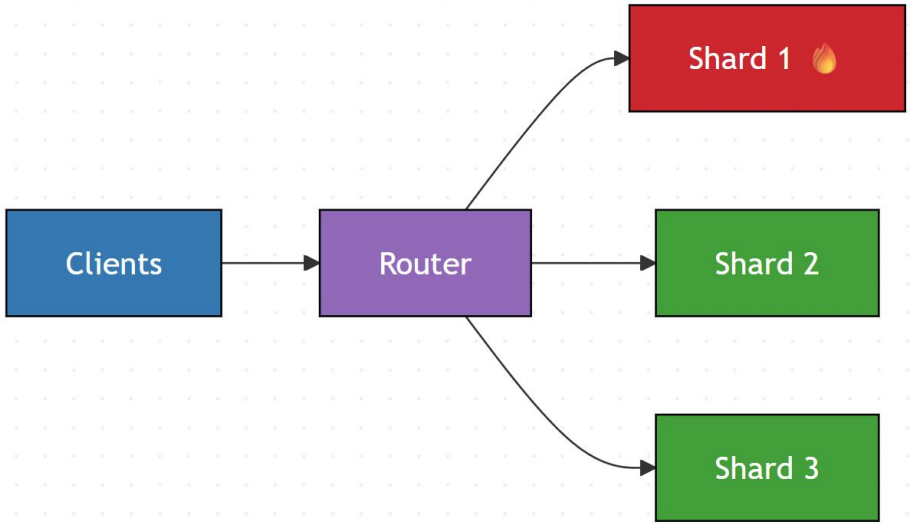
Sharding

- Horizontal data partitioning
- Each shard handles subset
- Scales reads and writes
- Requires shard key choice
- Cross-shard queries expensive



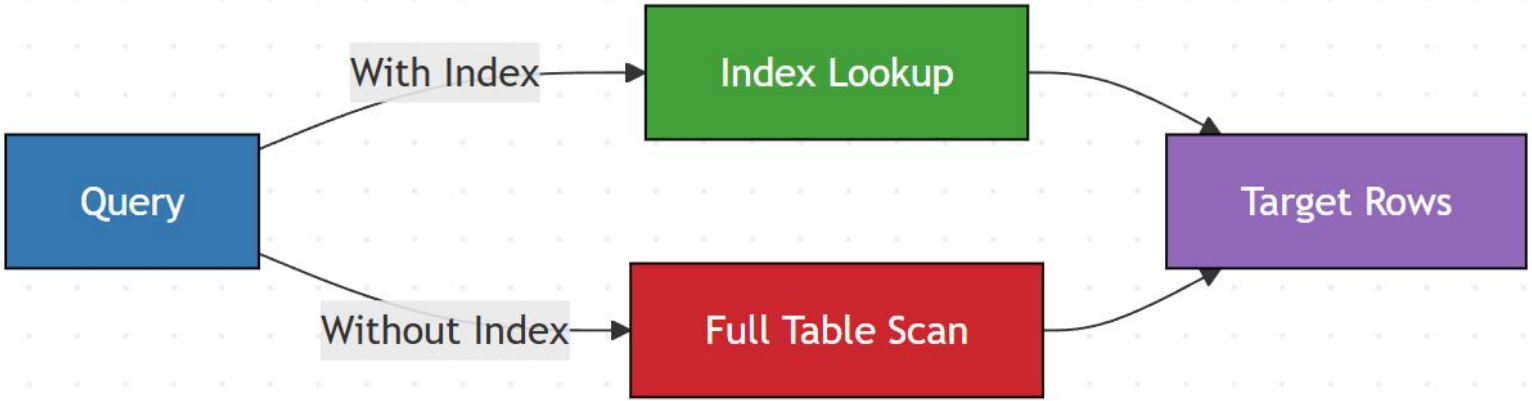
Shard Key Selection Pitfalls

- Uneven traffic causes hot shards
- Time-based keys overload latest shard
- Low-cardinality keys don’t distribute
- Cross-shard queries increase latency
- Resharding is expensive and risky



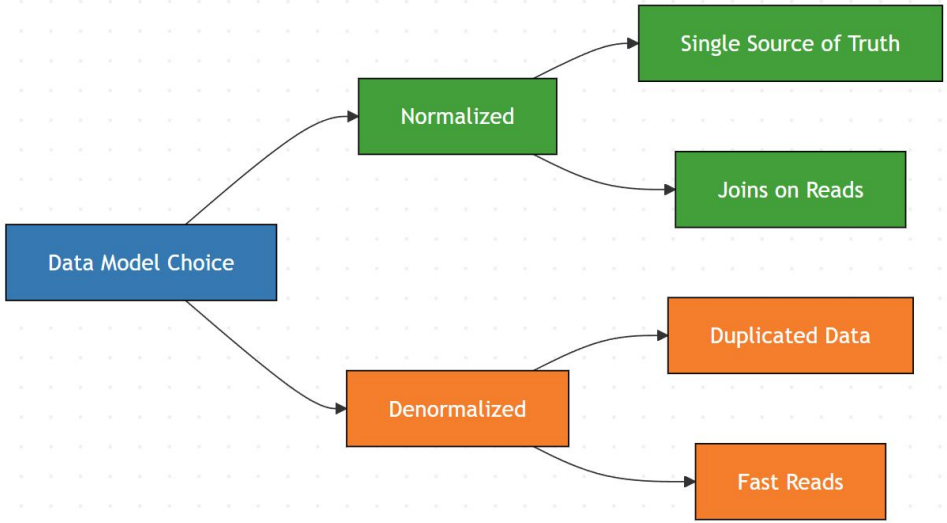
Indexing

- Index speeds up read queries
- Extra structure beside main data
- Improves reads, slows writes
- Consumes memory and storage
- Wrong indexes hurt performance



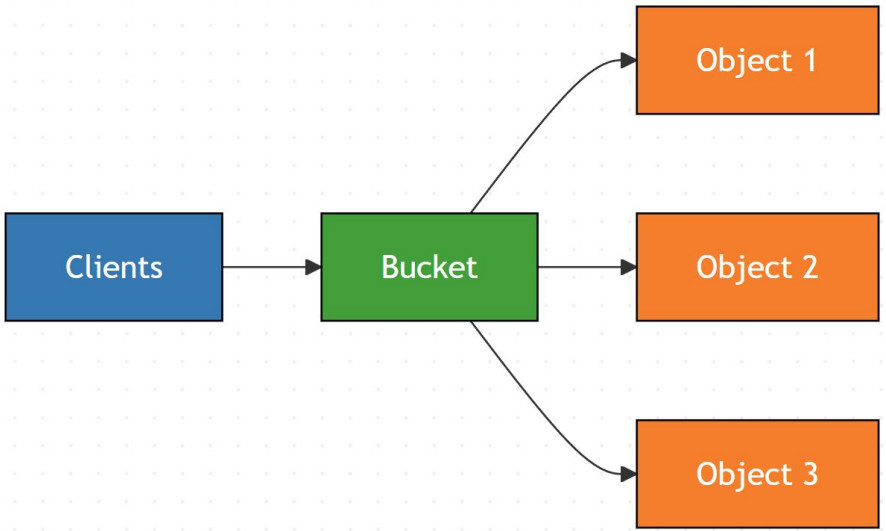
Normalization vs Denormalization

- Normalization reduces data duplication
- Denormalization duplicates for faster reads
- Joins vs precomputed data
- Normalized favors writes
- Denormalized favors reads



Object Storage

- Data stored as objects
- Buckets hold unlimited objects
- Flat namespace, no directories
- Massive durability and scale
- Higher latency than block storage



File Storage vs Object Storage

- File storage uses directories
- Object storage uses buckets
- POSIX access vs API access
- File favors low latency
- Object favors massive scale